

Bitcoin Proof of Compute (BTCPC)

A Sovereign Blockchain for Verifiable AI Inference, Secured by Useful Work

Shin Devlin Version 0.3 — March 2026

Abstract

Bitcoin proved that a decentralized network, secured by proof of work, can create and maintain digital scarcity. But Bitcoin's work is purpose-agnostic — SHA-256 puzzles that produce nothing except security. Meanwhile, the world's demand for AI compute grows exponentially, concentrated in the hands of a few corporations.

Bitcoin Proof of Compute (BTCPC) is a sovereign blockchain that applies Bitcoin's core insight — that costly, verifiable work can secure a network and back a scarce asset — to a problem that matters: AI inference. Miners earn BTCPC by providing real AI compute to the network. The work that secures the network is the work that users actually want to buy.

BTCPC introduces two innovations beyond Bitcoin: **Proof of Compute** replaces abstract hash puzzles with useful AI inference, and **Cross-Chain Mining Rewards** automatically generate claimable tokens on every blockchain a miner links to their rig — creating multi-chain liquidity from the first block.

Total supply: **42,000,000 BTCPC** — the answer to life, the universe, and everything.

1. The Problem

1.1 Centralized AI Compute

Today, AI inference is controlled by a handful of companies: OpenAI, Google, Anthropic, Meta. This creates:

- **Single points of failure** — one API outage affects millions
- **Censorship risk** — providers decide what can and cannot be computed
- **Rent extraction** — monopoly pricing on compute that costs fractions of a cent
- **Privacy violation** — every prompt passes through corporate servers

1.2 Wasted Proof of Work

Bitcoin's network consumes approximately 150 TWh of electricity annually — more than many countries. This energy produces nothing except SHA-256 hashes. The security model works, but the computational output serves no secondary purpose.

1.3 Siloed Liquidity

Every new blockchain launches with zero liquidity. Projects spend millions bootstrapping trading pairs, market makers, and exchange listings. Cross-chain bridges are complex, expensive, and frequent targets of exploits. There is no mechanism for a new chain to generate liquidity on existing chains organically.

1.4 The Opportunity

What if the energy spent securing a network also produced something people want? What if mining on one chain automatically created liquidity on every other chain the miner was connected to? What if the act of doing useful work generated value across the entire crypto ecosystem simultaneously?

2. The BTCPC Blockchain

2.1 Sovereign Chain

BTCPC is its own blockchain — not a Layer 2, not a sidechain, not a token on another platform. Like Bitcoin, it has:

- **Its own genesis block**
- **Its own P2P network** for transaction and block gossip
- **Its own consensus mechanism** (Proof of Compute)
- **Its own mempool** for pending transactions and inference requests
- **Its own account-based ledger** tracking balances, stakes, and mining rewards
- **Its own block format** containing transactions, inference proofs, and epoch commitments

2.2 Block Structure

```
Block {
  header: {
    version,
    previous_block_hash,
    merkle_root_transactions,
    merkle_root_compute_proofs,
    timestamp,
    epoch_number,
    difficulty,
    miner_id
  },
  transactions: [
    { type: "transfer", from, to, amount, signature },
    { type: "stake", account, amount, signature },
    { type: "inference_request", requester, model, prompt_hash, fee },
    { type: "inference_result", node, request_id, result_hash, tokens_generated },
  ]
}
```

```

    { type: "claim_generate", miner, epoch, chain, amount, proof },
    ...
  ],
  compute_proofs: [
    { node_id, inference_count, tokens_generated, model, commitment_hash }
  ]
}

```

2.3 Account Model — Hive-Style Key Hierarchy

Every BTCPC account has four key pairs, derived from a single master password — identical to the Hive blockchain's proven account model:

12-Word BIP-39 Mnemonic

```

├── m/44'/btcpc'/0'/0/0 → Owner Key
├── m/44'/btcpc'/0'/1/0 → Active Key
├── m/44'/btcpc'/0'/2/0 → Posting Key
└── m/44'/btcpc'/0'/3/0 → Memo Key

```

2FA Password (separate from mnemonic)

```

└── pbkdf2(password + account, 100000 rounds) → 2FA Key

```

Account creation generates a standard BIP-39 12-word mnemonic. The mnemonic derives all four role keys via BIP-44 derivation paths. This means any BIP-39 compatible hardware wallet (Ledger, Trezor, etc.) can store and sign with the role keys — no custom firmware required.

The 2FA password is set separately and derives its own key. The mnemonic and password together form complete account control. Losing either one alone is not enough to steal funds.

Key	Permission Level	What It Can Do	Where It Lives
Owner	Highest	Change password, rotate all keys, recover account, update account authority	Cold storage (paper, hardware wallet, safe)
Active	Financial	Send BTCPC, stake/unstake, claim cross-chain rewards, set up recurring payments, manage multi-sig	Daily-use device, encrypted with TOTP

Key	Permission Level	What It Can Do	Where It Lives
Posting	Operational	Submit epoch commitments, register/update mining node, serve inference requests, sign compute proofs	Mining rig (can mine but CANNOT move funds)
Memo	Communication	Encrypt/decrypt private messages, encrypt inference request/result payloads between nodes	Any device needing private communication

Key properties:

- **Password changes rotate all derived keys instantly.** If your active key is compromised, change your password with the owner key. All four derived keys rotate. The attacker's stolen key is now worthless.
- **The posting key lives on your mining rig.** It can do everything mining requires but cannot transfer a single token. This means a compromised mining server cannot steal your funds.
- **The memo key enables private inference.** Users can encrypt prompts so only the assigned compute node can read them. Results are encrypted back. No one — not even other validators — sees the actual content.
- **Owner key is used rarely and stored cold.** Like Hive, responsible operators should never need their owner key in normal operations.

2.3.1 Protocol-Level Multi-Factor Authentication

BTCPC is the first blockchain with **protocol-enforced multi-factor authentication**. Every account has a configurable authentication profile that validators enforce on every transaction. This is not wallet-level security — it is consensus-level.

Two additional authentication factors are available:

Factor A — Password: - User sets a 2FA password (can differ from the master key-derivation password) - `2fa_private_key = pbkdf2(2fa_password, account_name, 100000 rounds)` - `2fa_public_key` stored on-chain - Transaction includes a signature from the 2FA key - Password changeable at any time via Owner key (rotates the 2FA keypair) - Validators verify the 2FA signature against the stored public key - The password never touches the chain — only signatures

Factor B — TOTP (Google Authenticator): - User scans a QR code to set up standard TOTP in any authenticator app - `totp_commitment = hash(totp_seed)` stored on-chain - Every 30 seconds, the TOTP seed + time window derives a keypair - Transaction includes: TOTP signature + time_window - Validators derive the expected public key from `totp_commitment + time_window` and verify - Accepts ± 1 window (90-second

tolerance, standard TOTP) - The TOTP secret never touches the chain — only the commitment and signatures

User-configurable authentication profiles:

Profile	Factors Required	Use Case
Key only	Role key signature	Minimum security (not recommended)
Key + Password	Role key + password-derived signature	Standard security
Key + TOTP	Role key + Google Auth signature	Convenient security
Key + Password + TOTP	All three signatures	Maximum security (3FA)

Users configure their profile via AccountUpdate (requires Owner key). Each profile applies to all Active and Owner key operations. Posting key operations (mining, epoch commits) can be configured separately — miners may want posting-key transactions to require fewer factors for automated operation.

Transaction format:

```
Transaction {
  type: "transfer",
  from: "satoshinakamoto",
  to: "alice",
  amount: "100.00000000 BTCPC",

  primary_signature: <signed with active key>,

  auth_factors: {
    password_signature: <signed with password-derived key>, // if enabled
    totp_signature: <signed with TOTP-derived key>,           // if enabled
    totp_time_window: 1743120000                               // if TOTP enabled
  }
}
```

Validator checks:

1. Is primary_signature valid for the "from" account's active key? ✓
2. What auth profile does this account require?
3. If password enabled: is password_signature valid? ✓
4. If TOTP enabled: is totp_signature valid for this time_window? ✓
5. All required factors present → accept transaction

Why no other chain has this:

- Bitcoin/Ethereum: single key, no recovery, no 2FA
- Hive: hierarchical keys but single-signature per transaction
- BTCPC: hierarchical keys + protocol-enforced 2FA/3FA with user choice

A compromised private key alone is not enough to steal funds. An attacker would also need the user's password and/or TOTP device. This is the security model users expect from banking — applied to a decentralized blockchain for the first time.

2.3.2 Hardware Wallet Integration

BTCPC's 2FA design works with **any BIP-39 hardware wallet out of the box** — no custom apps, no firmware modifications, no approval processes.

The role keys (owner/active/posting/memo) are derived from the 12-word mnemonic via standard BIP-44 paths. Any hardware wallet that supports BIP-39 can store and sign with these keys. The 2FA factor is handled separately by the CLI or wallet software:

```
$ btcpc-cli transfer --to alice --amount 100
```

```
[Hardware wallet requests confirmation – press button, enter PIN]
→ Signature 1: role key (from hardware wallet)
```

```
Enter password: *****
→ Signature 2: 2FA key (derived from password, computed by CLI)
```

Transaction broadcast with both signatures.

The hardware wallet handles what hardware wallets do best — securely storing keys and requiring physical confirmation. The CLI handles the 2FA password/TOTP separately. Two independent factors, two independent devices, zero custom integration required.

Supported configurations:

Setup	Factor 1 (role key)	Factor 2 (2FA)
Hardware wallet + password	Ledger/Trezor signs	Type password in CLI
Hardware wallet + TOTP	Ledger/Trezor signs	Enter Google Auth code in CLI
Software wallet + password	Local key signs	Type password in CLI
Software wallet + TOTP	Local key signs	Enter Google Auth code in CLI
Hardware wallet + password + TOTP	Ledger/Trezor signs	Password AND Google Auth (3FA)

2.3.3 Account Recovery

If a user loses access to their 2FA (forgotten password, lost TOTP device) but still has their 12-word mnemonic:

1. Submit a **recovery request** using the Owner key (this is the ONE transaction type that bypasses 2FA)
2. **72-hour time-lock** begins — the recovery request is announced on-chain
3. During the 72 hours, if the real owner (with valid 2FA) submits a **contest transaction**, the recovery is blocked and the attacker's attempt fails
4. After 72 hours with no contest — 2FA resets, user sets a new password/TOTP

This mirrors how banks handle lost 2FA credentials: a delay period with the opportunity to intervene. The 72-hour window gives the real owner time to notice and stop unauthorized recovery attempts.

2.4 Purpose-Built Contracts

BTCP does not support arbitrary smart contracts. Like Bitcoin's Script, BTCP has a limited set of purpose-built contract types hardcoded into the protocol:

Contract Type	Required Key	Purpose
Transfer	Active	Move BTCP between accounts
Staking	Active	Lock/unlock BTCP for mining eligibility
Escrow	Active	Hold BTCP for inference payment until delivery confirmed
Slashing	Protocol	Automatically slash stakes for consensus violations
Claim	Active	Generate signed proofs for cross-chain reward claiming
MultiSig	Active	M-of-N approval requirement on an account
RecurringPay	Active (once)	Authorized periodic transfers to a target wallet
AccountUpdate	Owner	Change password, rotate keys, update authorities

Contract Type	Required Key	Purpose
NodeRegister	Posting	Register/update mining node configuration
EpochCommit	Posting	Submit epoch work commitment

No Solidity. No EVM. No arbitrary code execution. This is a deliberate design choice: the BTCPC chain does one thing — verifiable compute — and does it securely. Developers who want full smart contract capability use wrapped BTCPC (wBTCPC) on Base, Ethereum, or any EVM chain.

2.4 Epochs

Time in BTCPC is measured in **epochs**. One epoch = 5 minutes (~60 blocks at 5-second intervals).

At each epoch boundary: 1. Nodes submit their epoch commitment: {state_hash, tx_count, inference_count, proof_samples[] } 2. The network reaches consensus on the canonical state 3. Block rewards are distributed to honest miners 4. Cross-chain claim proofs are generated for linked wallets 5. Difficulty adjusts if threshold reached

2.5 The Work Function

Unlike Bitcoin's single work function (SHA-256), BTCPC has a composite work function:

$$\text{Work} = \sum(\text{inference_tokens_generated} \times \text{model_weight_factor})$$

Where model_weight_factor scales with model size: - 1B-7B parameters: 1.0x - 7B-13B parameters: 2.0x - 13B-30B parameters: 4.0x - 30B-70B parameters: 8.0x - 70B+ parameters: 16.0x

Running a 70B model earns 16x more than a 7B model per token generated — reflecting the real compute cost difference.

3. Verification: Proving Compute Is Real

The core challenge: how do you verify that a node actually performed the inference it claims, without re-running every computation?

3.1 The Verification Trilemma

Any verification system must balance: - **Security** — cheaters must be caught - **Efficiency** — verification must cost less than the work itself - **Speed** — verification must not bottleneck the network

BTCP achieves 100% verification through redundant computation with a cryptographic commit-reveal scheme.

3.2 Redundant Computation with Commit-Reveal

Every inference request is assigned to multiple nodes simultaneously. A cryptographic commit-reveal scheme prevents result copying.

Phase 1 — Commit: 1. Network assigns the same inference request to N nodes (N=3 for standard, N=5 for high-value) 2. Each node runs the inference independently with deterministic parameters (temperature=0, fixed seed derived from epoch + request hash) 3. Each node encrypts their result and submits only the **hash** to the network 4. No node can see another's result before committing — copying is impossible

Phase 2 — Reveal: 1. Once all N hashes are submitted, nodes reveal their actual results 2. Results are compared against the committed hashes (no post-hoc modification possible) 3. Matching results form consensus — this IS the verified answer 4. Non-matching nodes are immediately identified

Payment distribution:

First node to submit matching hash:	50% of request fee
Second matching node:	30% of request fee
Third matching node:	20% of request fee
Non-matching nodes:	slashed

This creates a **race** — nodes compete to finish first, incentivizing better hardware. The fastest honest miner earns the most, just like Bitcoin miners racing to find a hash.

Scaling N with network size: - Genesis (solo miner): N=1 (no redundancy needed) - Early network (2-5 nodes): N=all nodes (everyone verifies everything) - Growth phase (5-50 nodes): N=3 (standard redundancy) - Mature network (50+ nodes): N=3 standard, N=5 for high-value requests

3.3 Physical Plausibility Bounds

Nodes declare their hardware capabilities upon registration: - GPU model, VRAM, count - CPU cores, RAM - Maximum concurrent inferences

The network maintains benchmark data for known hardware configurations. A node claiming to generate 1000 tokens/second on a single RTX 4090 with a 27B parameter model is physically impossible (~15-30 tokens/second is realistic). Claims that violate plausibility bounds trigger automatic verification of all that node's work for the epoch.

3.4 Slashing Protocol

When a node's result doesn't match consensus:

1. First offense: warning + reputation penalty
2. Second offense within 1000 epochs: **10% of staked BTCP slashed**
3. Third offense: **25% slashed + 24-hour mining suspension**

4. Persistent offenders: **full stake slashed + permanent ban**

Slashed BTCPC is redistributed to the honest nodes who produced the correct result.

3.5 Why This Works (Game Theory)

For a rational miner with stake S and epoch earnings E :

Expected value of honest mining = E

Expected value of cheating = $E \times \text{savings_from_faking} - S \times 0.25 \times P(\text{caught})$

With commit-reveal redundant computation, $P(\text{caught}) = 1.0$ for any request assigned to multiple nodes. Since $S \gg E$ (minimum stake requirement ensures this), cheating is always negative expected value. The system is **incentive-compatible** — honest behavior is the dominant strategy regardless of what other miners do.

4. Inference Protocol: End-to-End Encrypted Compute

4.1 Mandatory Encryption

All inference on the BTCPC chain is end-to-end encrypted. This is not optional. No plaintext prompt or result ever touches the chain or is visible to validators, other nodes, or the public.

On-chain (public):	Off-chain (private):
prompt_hash (sha256)	actual prompt (plaintext)
encrypted_prompt (memo key)	actual result (plaintext)
result_hash (sha256)	decryption keys
encrypted_result (memo key)	
model, tokens_used, fee, timestamp	

4.2 Encryption Flow

1. User submits inference request:
 - Encrypts prompt with ASSIGNED NODE's memo public key
 - Includes prompt_hash = sha256(plaintext_prompt)
 - Posts: {encrypted_prompt, prompt_hash, model, fee}
2. Network assigns request to N nodes for commit-reveal:
 - Each node decrypts the prompt using their memo private key
 - Runs inference with deterministic parameters
 - Computes result_hash = sha256(plaintext_result)
 - Encrypts result with REQUESTER's memo public key
 - Commits result_hash to the network
 - Discards the decrypted prompt from memory
3. Reveal phase:
 - Nodes reveal result_hashes

- Consensus determined by matching hashes
- Winning encrypted_result delivered to requester
- Requester decrypts with their own memo private key

4. What remains on-chain:

- prompt_hash, result_hash (for verification audit trail)
- encrypted_prompt, encrypted_result (unreadable without keys)
- model, tokens, fee, timestamp (operational metadata)
- NO plaintext. Ever.

4.3 Privacy Guarantees

Party	Can See Prompt?	Can See Result?
Requester	Yes (they wrote it)	Yes (they decrypt it)
Assigned compute nodes	Temporarily (decrypt to process, then discard)	Temporarily (generate, then discard after hashing)
Other nodes / validators	No (only see hashes)	No (only see hashes)
Public / block explorer	No	No
Chain operator / foundation	No	No

4.4 Comparison with Centralized AI

Property	OpenAI / Google / Anthropic	BTCP
Who reads your prompt	The company, their employees, their training pipeline	Only the compute node, temporarily
Prompt storage	Stored on corporate servers indefinitely	Encrypted on-chain, only readable by requester
Data used for training	Often yes (unless opted out)	Never — nodes discard plaintext after processing

Property	OpenAI / Google / Anthropic	BTCP
Censorship	Provider decides what you can ask	No censorship — encrypted, no one can read to censor
Subpoena risk	Company can be compelled to hand over logs	No plaintext exists to hand over

4.5 Network Privacy

Beyond encrypted inference content, BTCP provides three privacy layers at the network level:

Anonymous Inference Routing: Users can submit inference requests through a **relay mixer**. The request is routed through one or more relay nodes that strip the sender's identity before forwarding to the compute node. The compute node sees only: a valid escrow, an encrypted prompt, and a return path through the relay. It does not know who is asking.

User → Relay A → Relay B → Compute Node

User ← Relay A ← Relay B ← Result

Compute node sees: anonymous request + valid escrow

User sees: result

Relays see: encrypted traffic (cannot read prompt or result)

Nobody can link the request to the user

Relay nodes earn a small fee (1-2% of the inference fee) for providing this service. Anonymous routing is optional — users choose per-request whether to route directly or through relays.

Node IP Privacy (Tor/I2P Integration): Mining nodes can optionally expose their API endpoint as a Tor hidden service or I2P address instead of a clearnet IP. This means:

- No one can determine the physical location of a compute node
- ISPs cannot see that a machine is running a BTCP node
- The node operator's identity is protected

Standard node: `https://203.0.113.42:8080` (IP visible)

Private node: `http://abc123...xyz.onion` (location hidden)

Node-to-node communication (block gossip, peer discovery) can also route through Tor, making the entire P2P network invisible to network-level surveillance.

Stealth Accounts: For users who need stronger privacy than usernames provide, BTCP supports **stealth addresses**. A stealth address is a one-time account derived from the recipient's public key that cannot be linked back to their main account:

Alice's main account: alice
Alice's stealth address: s.7f3a9b2c... (one-time, unlinkable)

Anyone can send BTCPC to s.7f3a9b2c
Only Alice can spend from it (derived from her keys)
No one can prove s.7f3a9b2c belongs to alice

Each transaction can generate a new stealth address. An observer watching the blockchain sees transfers between seemingly unrelated one-time accounts — they cannot determine that the same person is behind multiple transactions.

Privacy Summary:

Layer	What It Hides	Default
Encrypted inference	Prompt + result content	Always on (mandatory)
Anonymous routing	Who submitted the request	Optional per-request
Tor/I2P nodes	Where nodes are physically located	Optional per-node
Stealth accounts	Which transactions belong to the same person	Optional per-transaction

Future (Phase 4): Full Confidential Transactions — hidden amounts, ring signatures, zero-knowledge proofs. Monero-level privacy for users who need it.

4.6 Submitting Inference Requests

Layer 1 — On-chain transaction (raw):

```
{
  "type": "inference_request",
  "requester": "alice",
  "model": "qwen3.5:27b",
  "prompt_hash": "sha256(prompt)",
  "encrypted_prompt": "<encrypted with node memo key>",
  "max_fee": "10 BTCPC",
  "primary_signature": "<active key>",
  "auth_factors": { "password_signature": "<2fa>" }
}
```

BTCPC held in escrow until delivery confirmed.

Layer 2 — Node API (practical, OpenAI-compatible):

POST <https://node.btcpc.network/v1/inference>
Authorization: Bearer btcpc_apikey_...

```
{
  "model": "qwen3.5:27b",
```

```
"messages": [{"role": "user", "content": "What is 6 times 7?"}]
}
```

- Node handles encryption, on-chain escrow, verification
- Returns decrypted result to the API caller
- Feels identical to calling OpenAI

Layer 3 — SDK:

```
const btcpc = require('btcpc-sdk');
const client = new btcpc.Client({ apiKey: 'btcpc_...' });
const res = await client.inference({
  model: 'qwen3.5:27b',
  prompt: 'Explain quantum computing'
});
// Encrypted end-to-end. Paid in BTCPC. Verified on-chain.
```

4. Token Economics

4.1 Supply

Total supply: 42,000,000 BTCPC

42 — the Answer to the Ultimate Question of Life, the Universe, and Everything, as computed by Deep Thought over 7.5 million years in Douglas Adams' *The Hitchhiker's Guide to the Galaxy*. Deep Thought was, arguably, the first proof of compute system: a massive computer that ran for millennia to produce a single, verified answer.

BTCPC is the modern incarnation of that idea: a global network of computers performing useful computation, producing verified answers, and earning tokens for their work. The total supply honors the original answer.

4.2 Emission Schedule — Doubling Halving Intervals

BTCPC introduces a novel emission model: **doubling halving intervals**. Each period, the block reward halves while the period duration doubles. Unlike Bitcoin's fixed 4-year halvings, BTCPC's emission curve adapts: early periods distribute tokens quickly to bootstrap the network, while later periods extend over decades, ensuring long-term miner incentives.

With a genesis allocation of 5% of total supply (2,100,000 BTCPC) in the first 1-month period, and a growth ratio of $\sim 1.134x$ per period, each subsequent period's total allotment grows slightly in absolute terms but the per-epoch reward decreases. This means the inflation RATE drops consistently while the network can support an ever-growing number of miners.

Period	Duration	Allotment	Reward/Epoch	Cumulative	% Supply	Annual Inflation
1	1 month	2,100,000	243.06	2,100,000	5.0%	∞ (genesis)
2	2 months	2,381,400	137.85	4,481,400	10.7%	High (early)
3	4 months	2,700,508	78.19	7,181,908	17.1%	~87%
4	8 months	3,062,376	38.01	10,244,283	24.4%	~29%
5	16 months	3,472,734	25.15	13,717,017	32.7%	~11%
6	32 months	3,938,080	14.24	17,655,097	42.0%	~4.8%
7	64 months	4,465,783	8.08	22,120,881	52.7%	~2.2%
8	128 months	5,064,198	4.58	27,185,079	64.7%	~1.0%
9	256 months	5,742,801	2.60	32,927,879	78.4%	~0.5%
10	512 months	6,512,336	1.47	39,440,215	93.9%	~0.2%
11	345 months*	2,559,785	0.86	42,000,000	100%	~0.1%

*Period 11 is truncated to align final mining with Bitcoin's projected end date of ~2140.

Key properties: - **Genesis month (solo mining):** 5% of supply — earned through real compute - **By year 5:** inflation drops to ~11%, comparable to Bitcoin's era 2 - **By year 10:** inflation at ~5%, matching BTC's era 3 - **All 42M mined by ~2140** — same timeline as Bitcoin's last satoshi - **No period dominates:** unlike fixed halvings where era 1 gets 50%, the largest single period (period 10) gets only 15.5% - **Growing allotments reward growing networks:** more miners in later periods have more tokens to compete for

4.3 Reward Distribution Per Epoch

Each epoch's block reward is distributed proportional to verified work:

$$\text{miner_reward} = \text{block_reward} \times (\text{miner_work} / \text{total_network_work})$$

$$\text{miner_work} = \sum(\text{tokens_generated} \times \text{model_weight_factor}) \text{ [verified only]}$$

During the genesis period (solo mining), 100% of the block reward goes to the single miner. As more nodes join, the reward distributes naturally.

4.4 Difficulty Adjustment

Every 1,000 epochs (~3.5 days), the network adjusts difficulty:

$\text{new_difficulty} = \text{old_difficulty} \times (\text{actual_work} / \text{target_work})$
 $\text{target_work} = \text{expected tokens across all nodes at current difficulty}$

If more GPUs join the network → each unit of compute earns fewer BTCPC → mirrors Bitcoin's difficulty adjustment exactly.

4.5 Fee Market

Users pay for inference in BTCPC: - **Base fee** = network-determined minimum per 1K tokens (adjusts with demand) - **Priority fee** = optional tip for faster processing - **Miner revenue** = block reward + 100% of fees + priority fees - **No burning** — every BTCPC minted stays in circulation forever, like Bitcoin

As block rewards diminish through halvings, fee revenue becomes the primary miner incentive — exactly like Bitcoin's long-term security model.

5. Cross-Chain Mining Rewards

5.1 The Innovation

BTCPC introduces a mechanism never before implemented in cryptocurrency: **cross-chain mining rewards**. When a miner links wallets from other blockchains to their BTCPC mining node, they automatically earn claimable wrapped BTCPC (wBTCPC) tokens on every linked chain — simply by mining on the BTCPC chain.

This is not bridging. This is not wrapping. This is **simultaneous multi-chain value creation from a single act of compute**.

5.2 How It Works

Miner registers linked wallets:

BTCPC:	btcpclabc...	(native, always)
Hive:	@satoshinakamoto	(linked)
Base:	0x1234...	(linked)
Solana:	ABC123...	(linked)

Miner performs compute in epoch 1, earns 243 BTCPC:

BTCPC chain:	+243.0	BTCPC	← native reward (always)
Hive:	+243.0	wBTCPC	← cross-chain bonus (claimable)
Base:	+243.0	wBTCPC	← cross-chain bonus (claimable)
Solana:	+243.0	wBTCPC	← cross-chain bonus (claimable)

The cross-chain bonus starts at 1:1 parity with the native reward and halves with each emission period:

Period	Duration	Native Reward	Cross-Chain Ratio	wBTCPC per Linked Chain
1	1 month	243.06 / epoch	1:1 (100%)	243.06 / epoch
2	2 months	137.85 / epoch	1:2 (50%)	68.93 / epoch
3	4 months	78.19 / epoch	1:4 (25%)	19.55 / epoch
4	8 months	38.01 / epoch	1:8 (12.5%)	4.75 / epoch
5	16 months	25.15 / epoch	1:16 (6.25%)	1.57 / epoch
6	32 months	14.24 / epoch	1:32 (3.125%)	0.45 / epoch
...	...	...	...	...

5.3 Key Rules

1. **Wallet must be linked at time of compute.** You cannot retroactively claim rewards for epochs before your wallet was linked. This prevents gaming — you must commit to the ecosystem to benefit.
2. **Claiming is the miner's responsibility.** The BTCPC chain generates a signed claim proof each epoch. The miner takes this proof to the target chain and submits it to the wBTCPC claim contract. The miner pays any fees on the target chain (Hive RC, Base gas, etc.). BTCPC pays nothing.
3. **One claim per epoch per chain.** Each epoch's reward can only be claimed once on each chain. The claim contract tracks claimed epochs.
4. **wBTCPC is freely tradeable** on each chain. Once claimed, wBTCPC-Hive trades on Hive DEXs, wBTCPC-Base trades on Base DEXs, etc. Each has its own independent market price.
5. **No cap on linked chains.** A miner can link as many chains as BTCPC supports. More linked chains = more total value earned per epoch of compute. This incentivizes miners to expand the BTCPC ecosystem.

5.4 Why This Works

For miners: Linking wallets is pure upside. Same compute work, more rewards. Every rational miner links every available chain.

For the BTCPC ecosystem: Every linked wallet means wBTCPC tokens appearing on another chain. This creates organic liquidity, market presence, and awareness on chains BTCPC never had to launch on. The miners themselves bootstrap the multi-chain ecosystem.

For other chains: They get a new tradeable token (wBTCPC) backed by real compute work, with zero effort. It just appears because miners linked their wallets.

Economic sustainability: The 1:1 ratio in period 1 is aggressive but limited — only 1 month of mining at that rate, with only 1-2 miners. By period 4, the ratio is 12.5%. By period 6, it's 3.125%. The cross-chain supply growth naturally decays while the native chain's value proposition strengthens.

5.5 Claim Proof Format

```
{
  "chain": "base",
  "miner": "btcpc1abc...",
  "target_wallet": "0x1234...",
  "epoch": 42,
  "amount": "243.06000000",
  "period": 1,
  "cross_chain_ratio": "1.0",
  "linked_at_epoch": 0,
  "proof": "0x[SIGNED_BY_BTCPC_CONSENSUS_NODES]"
}
```

The claim contract on each chain verifies: 1. The proof signature is valid (signed by BTCPC consensus) 2. This epoch has not been claimed on this chain before 3. The target wallet matches the linked wallet in the proof 4. The amount matches the cross-chain ratio for the period

5.6 Supported Chains (Day 1 and Beyond)

Day 1: - Hive (wBTCPC as custom_json token) - Base (wBTCPC as ERC-20)

Planned: - Solana (wBTCPC as SPL token) - Arbitrum / Optimism - TON - Any EVM-compatible chain via standardized claim contract

Adding a new chain requires only deploying a wBTCPC claim contract on that chain. No changes to the BTCPC core protocol.

6. Genesis: The shindevlin Epoch

6.1 Solo Mining Phase

BTCPC launches with a single miner: **shindevlin** — running a GPU-equipped machine running Ollama with multiple AI models.

This mirrors Bitcoin's genesis: - Satoshi Nakamoto mined alone for months before anyone else joined - The early blocks were nearly worthless - The work was real, the tokens accumulated, and value emerged later

Genesis configuration: - Models: qwen3.5:27b, deepseek-r1:8b, glm-4.7-flash - Epoch interval: 5 minutes - Initial block reward: 243 BTCPC per epoch - Period 1 output: 2,100,000 BTCPC over 1 month (5% of total supply) - Daily mining output: ~69,984

BTCPC (288 epochs × 243 BTCPC) - Linked chains: Hive, Base - Cross-chain bonus: 1:1 per linked chain

6.2 Genesis Block

Block 0 – BTCPC Genesis

Timestamp: [TBD]

Miner: shindevlin

Models: qwen3.5:27b

State Hash: 0x00

Message: "The Answer to the Ultimate Question of Life, the Universe, and Everything"

Reward: 243 BTCPC

Cross-Chain: 243 wBTCPC-Hive (claimable), 243 wBTCPC-Base (claimable)

6.3 Opening the Network

When the first external node operator wants to join: 1. They install the BTCPC node software 2. They create a BTCPC account on the native chain 3. They stake a minimum of 1000 BTCPC (received via transfer from genesis miner or purchased) 4. They link wallets for any chains they want cross-chain rewards on 5. They begin serving inference and earning rewards

The staking requirement creates a bootstrapping challenge: early nodes need BTCPC to stake, but BTCPC only comes from mining. This is solved by: - **Genesis grants**: the solo miner distributes initial stakes to early node operators - **Faucet**: a small amount available for new nodes to bootstrap - **Delegation**: existing stakers can delegate stake to new nodes

7. Node Software

7.1 Reference Implementation

The reference BTCPC node is built in Node.js:

```
btcp-node start \  
  --ollama-url http://localhost:11434 \  
  --stake 1000 \  
  --link-hive @satoshinakamoto \  
  --link-base 0x1234...
```

7.2 Node Components

BTCPC Node		
P2P Layer	Inference Server	Epoch Worker

Block gossip, tx relay, peer discovery	(Ollama proxy) Serves requests, logs proofs	Computes state hash, submits commitments, generates claim proofs
State Manager (MongoDB) Accounts, Balances, Stakes, Blocks, Proofs		
Chain Manager Block production, mempool, chain sync		
Claim Proof Generator Signs cross-chain reward proofs for linked wallets per epoch		

7.3 Minimum Hardware Requirements

Inference tier (earns full rewards + cross-chain bonuses): - GPU with 8GB+ VRAM (runs 7B+ models) - 16GB RAM - 4 CPU cores - 100GB SSD - 50 Mbps internet

Relay tier (earns partial rewards, no inference): - No GPU required - 8GB RAM - 2 CPU cores - 50GB SSD - Transaction processing and state validation only - Earns ~10% of what inference nodes earn - Still earns cross-chain bonuses on linked wallets

8. Roadmap

Phase 0: Genesis (Current)

- ☒ Whitepaper
- ☒ Architecture specification
- ☒ Wallet controller (transfer, balance)
- ☒ Staking controller (stake, unstake, withdraw)
- ☒ Epoch system with emission schedule
- ☒ Node registration
- ☐ P2P network layer
- ☐ Block production and chain sync
- ☐ Genesis mining loop on shindevin

Phase 1: Solo Mining

- ☐ shindevin mining and accumulating BTCPC
- ☐ Cross-chain claim proof generation
- ☐ wBTCPC claim contract on Hive
- ☐ wBTCPC claim contract on Base (ERC-20)

- ☐ Mining dashboard (web UI)
- ☐ btcpc-cli tool

Phase 2: Network Opening

- ☐ Node registration protocol (live)
- ☐ Stake requirement enforcement
- ☐ Commit-reveal verification system (N=3)
- ☐ Difficulty adjustment
- ☐ Block explorer

Phase 3: Inference Market

- ☐ Public inference API (pay with BTCPC)
- ☐ Model marketplace (nodes advertise capabilities)
- ☐ Fee market and priority system
- ☐ Inference routing (match requests to best nodes)

Phase 4: Maturity

- ☐ First halving (period 2 begins)
- ☐ Additional chain support (Solana, Arbitrum, TON)
- ☐ Governance (stake-weighted voting)
- ☐ Mobile relay node support
- ☐ Formal security audit

9. Conclusion

Bitcoin proved that decentralized proof of work can create sound money. BTCPC extends this insight in two fundamental ways:

First, the work that secures the network is useful. Every BTCPC token is backed by real AI compute — inference that someone wanted, paid for, and received. Every unit of energy produces useful output.

Second, mining on one chain creates value on many. A single act of compute generates native BTCPC and claimable wBTCPC tokens on every linked blockchain. Miners don't just secure one network — they bootstrap liquidity across the entire crypto ecosystem.

The result is a network where: - **Miners earn by doing useful work** (directing energy toward useful computation) - **Users get censorship-resistant AI compute** (not dependent on any corporation) - **The token has intrinsic utility** (you need it to buy inference, creating organic demand) - **Supply is fixed and predictable** (42M total, doubling halving intervals, no burns) - **Multi-chain liquidity emerges organically** (miners create it by linking wallets)

The answer is 42. The question was always about compute.

"I think the problem, to be quite honest with you, is that you've never actually known what the question is." — Deep Thought, The Hitchhiker's Guide to the Galaxy

Appendix A: Comparison with Bitcoin

Property	Bitcoin	BTCP
Type	Sovereign chain	Sovereign chain
Total Supply	21,000,000	42,000,000
Work Function	SHA-256 (security-only)	AI Inference (useful)
Block Time	~10 minutes	~5 minutes (1 epoch)
Halving Interval	Fixed 4 years	Doubling intervals (1mo → 2mo → 4mo → ...)
Verification	Check hash (instant)	Commit-reveal redundant computation (100%)
Mining Hardware	ASICs	GPUs + AI models
Multi-Chain	Bitcoin chain only	Native + claimable wBTCP on linked chains
Smart Contracts	Script (limited)	Purpose-built (limited)
Fee Model	Transaction fees	Inference fees + transaction fees
Burns	None	None
Final Token Mined	~2140	~2140

Appendix B: Comparison with Compute Networks

Property	Render	Akash	io.net	BTCP
Chain Type	Token on Ethereum/Solana	Cosmos chain	Token on Solana	Sovereign chain
Token Model	Utility (inflationary)	Utility (inflationary)	Utility (inflationary)	Fixed supply (42M)
Mining	No mining	No mining	No mining	Yes — earn by providing compute
Consensus	None (marketplace)	Tendermint (PoS)	None (marketplace)	Proof of Compute
Verification	Trust-based	Trust-based	Trust-based	Commit- reveal (100%)

Property	Render	Akash	io.net	BTCP
Halvings	No	No	No	Yes (doubling intervals)
Multi-Chain Rewards	No	No	No	Yes (1:1 halving cross-chain)
Burns	No	No	No	No
AI-Native	No (general GPU)	No (general compute)	Partial	Yes (inference-first)

Appendix C: Cross-Chain Reward Supply Projections

Assuming genesis miner links 2 chains (Hive + Base) and the network grows to 100 miners by period 4, each linking an average of 3 chains:

Period	Native BTCP Minted	Cross-Chain Ratio	wBTCP per Chain (est.)	Total wBTCP All Chains
1	2,100,000	100%	2,100,000	4,200,000 (2 chains)
2	2,381,400	50%	1,190,700	2,381,400 (2 chains)
3	2,700,508	25%	675,127	2,025,381 (3 chains avg)
4	3,062,376	12.5%	382,797	1,148,391 (3 chains avg)
5	3,472,734	6.25%	217,046	868,183 (4 chains avg)

The cross-chain supply growth naturally decelerates while native BTCP remains the primary store of value.

Appendix D: Units and Inscriptions

The Dream

The smallest unit of BTCP is the **dream**.

1 BTCP = 100,000,000 dreams

Named for Deep Thought's 7.5-million-year computation — a machine dreaming the answer into existence. Every dream on BTCP represents a unit of verified computation: a machine dreaming an answer for someone who asked.

Genesis Dream Inscriptions

The first dream of every block — the **genesis dream** — can carry an inscription: arbitrary content permanently embedded in the chain. One inscription per block, immutable, forever.

Inscriptions are a purpose-built contract type:

```
{
  "type": "inscribe",
  "block": 42,
  "creator": "shindevlin",
  "data": {
    "type": "build",
    "name": "My Decentralized AI App",
    "description": "Built on BTCPC compute",
    "url": "https://myapp.com",
    "tags": ["ai", "inference", "privacy"]
  },
  "signature": "<active key>"
}
```

What makes BTCPC inscriptions unique: - Bitcoin Ordinals inscribe data on satoshis — arbitrary, no inherent meaning - BTCPC inscriptions sit on **dreams** — units of verified computation - Each inscribed dream represents real AI inference that actually happened - Builders inscribe what they’re creating WITH the compute — the inscription and the work are linked - The genesis dream of block 0 carries: *“The Answer to the Ultimate Question of Life, the Universe, and Everything”*

Every token is a dream computed into reality. Every block begins with a dream that can carry the builder’s inscription — a permanent record of what was imagined and built on this chain.

Appendix E: Proof of Silicon — GPU-Bound Inference Privacy

E.1 The Problem

Confidential computing (NVIDIA H100, AMD SEV) provides hardware-guaranteed privacy but requires expensive datacenter hardware. Consumer GPUs — the backbone of decentralized compute — have no memory encryption. A node operator with root access can theoretically read GPU VRAM.

E.2 Silicon Identity Key (SIK)

BTCPC introduces **Proof of Silicon**: a cryptographic identity derived from the physical manufacturing variations of a specific GPU die.

Every GPU has transistor-level imperfections unique to that chip. These are measurable, reproducible, and unclonable:

1. **VRAM Timing Probe** — Measures nanosecond-level read latency variations across thousands of memory cells. Each cell's electrical characteristics differ due to manufacturing variance.
2. **Floating-Point Divergence Probe** — Runs identical deterministic math on every GPU. Due to ALU transistor variations, the least significant bits of results differ between physical dies. Same code, different silicon, different bits.

Combined, these produce a 256-bit **Silicon Identity Key (SIK)** that is: - Unique per physical GPU - Reproducible across reboots - Not derivable from serial numbers - Impossible to replicate without possessing that exact chip

E.3 SIK-Bound Encryption

Inference session keys are derived from: $\text{HKDF}(\text{ECDH_shared_secret} + \text{SIK}, \text{session_id})$.

The SIK component means the decryption key **physically cannot exist** without the registered GPU. Copy the node's disk to another machine → different GPU → different SIK → wrong key → cannot decrypt.

E.4 Zero-Plaintext Inference Pipeline

Plaintext prompts never exist on the inference node — not even for microseconds:

1. User tokenizes the prompt **on their own device**
2. User generates a per-session random permutation of the token vocabulary
3. User remaps all token IDs through this permutation
4. User encrypts the remapped token IDs with the SIK-bound session key
5. Node decrypts → receives remapped integer arrays
6. GPU processes remapped tokens — VRAM contains only meaningless integers
7. Node encrypts output tokens, sends back to user
8. User de-remaps on their device

The node cannot reverse the remapped tokens to text because: - It does not have the remap permutation table - The token IDs don't correspond to the model's real vocabulary - Even dumping GPU VRAM yields random-looking integers

E.5 Remote Verification

The network can challenge any node to prove it's using its registered GPU:

Network → "Run SIK probe, sign result with posting key"

Node → derives SIK from physical GPU → signs → returns

Network → $\text{sha256}(\text{SIK})$ matches registered `sik_hash`?

Yes → same GPU, node is legitimate

No → GPU swapped or spoofed → slash stake

E.6 Node Privacy Tiers

Tier	Hardware	Privacy Level	Fee Multiplier
Silicon	Consumer GPU with SIK probe	Practical privacy — software isolation + token remapping	1.0x
Confidential	NVIDIA H100/H200 + SIK	Hardware-guaranteed — encrypted GPU memory + SIK	1.5x
Software	CPU-only, no GPU	Transport encryption only — no silicon binding	0.5x

Appendix F: Verification Evolution — Future-Proof Protocol

Current: Commit-Reveal Redundant Computation (N=3)

The current verification model assigns each inference request to N nodes. All N run the same computation independently. Consensus determines truth. This works but costs N_x the compute.

Future: Single-Pass Verifiable Inference

Research in verifiable computation, zero-knowledge machine learning (zkML), and model-level attestation is advancing rapidly. It is plausible that within the lifetime of BTCPC, methods will exist to verify that a neural network inference was performed correctly — without re-running it.

Potential mechanisms: - **zkML proofs** — zero-knowledge proofs that a specific model produced a specific output from a specific input, without revealing the input or re-running the model - **Trusted Execution Environments (TEEs)** — hardware attestation (Intel SGX, AMD SEV, NVIDIA Confidential Computing) that cryptographically proves specific code ran on specific hardware - **Model fingerprinting** — intermediate layer activations at checkpoint positions that can be verified cheaply without full re-execution - **Consensus-free verification** — mathematical properties of transformer attention patterns that prove computation integrity

Protocol Upgrade Path

The BTCPC protocol is designed so that the verification method is a **replaceable module**, not hardcoded into consensus:

Verification Interface:

```
verify(request, result, proof) → {valid: bool, confidence: float}
```

Current implementation:

```
CommitRevealVerifier (N=3 redundant computation)
```

Future implementations (hot-swappable via governance vote):

ZKMLVerifier	(zero-knowledge proof of inference)
TEEEVerifier	(hardware attestation)
HybridVerifier	(ZK for small models, commit-reveal for large)
SinglePassVerifier	(model-native verification, when available)

When a superior verification method becomes available: 1. A governance proposal is submitted with the new verifier implementation 2. Stake-weighted vote by node operators 3. If approved, the new verifier is activated at a specified epoch 4. Old proofs remain valid — verification is forward-compatible 5. N can be reduced to 1 — eliminating redundant computation costs entirely

This means BTCPC miners will eventually earn the FULL inference fee (not split N ways), making mining dramatically more profitable when single-pass verification arrives. Early miners who build the network now benefit from this future upgrade.

The protocol does not assume any specific verification technology. It assumes only that verified compute exists and is improvable. The chain adapts; the economic model endures.

Appendix F: Genesis Dreams and Mining Proofs

Two Artifacts Per Block

Every block on the BTCPC chain produces two distinct non-fungible artifacts:

1. Genesis Dream (transferable) The first dream of the block. A unique, inscribable, tradeable NFT. The miner who produced the block receives it and can: - Inscribe it with a dream inscription (once, permanently) - Transfer it to another account (sell, trade, gift) - Hold it as a collectible

Genesis dreams are the BTCPC equivalent of rare ordinal sats — but each one represents verified AI compute that actually happened.

2. Mining Proof (soulbound) A non-transferable badge permanently bound to the miner's account. Proves that this account produced this specific block. Cannot be bought, sold, faked, or transferred. It is soulbound to the miner forever.

Block 42 mined by shindevlin:

Reward: 243.05555556 BTCPC

Artifacts:

Genesis Dream #42 → transferable NFT (1 dream, locked from spendable balance)

Mining Proof #42 → soulbound badge (non-transferable, proves authorship)

Spendable: 243.05555555 BTCPC (reward minus 1 dream)

Genesis Dream Verification

Each genesis dream carries a cryptographic proof of authenticity that anyone can verify without trusting the holder:

```
Genesis Dream #42:
  block:          42
  original_miner: shindevlin
  timestamp:      2026-03-25T18:55:24.123Z
  state_hash:     abc123... (chain state at this block)
  work_hash:      def456... (hash of all inference work in this block)
  model:          qwen3.5:27b
  tokens_computed: 1536
  proof_signature: <signed by consensus>
  inscription:    { ... } (dream inscription, if inscribed)
```

Verification flow: 1. Check proof_signature against the chain's consensus key for block 42 2. Verify state_hash matches the canonical chain at block 42 3. Verify work_hash matches the recorded work proofs for block 42 4. Confirm original_miner matches the block producer record

Forgery is impossible — the proof signature chain goes back to genesis block 0. A fake genesis dream would fail signature verification instantly.

Even after transfer, the original_miner field is immutable. If shindevlin transfers Genesis Dream #42 to alice, alice owns the dream but shindevlin is permanently recorded as its creator. Provenance is built into the protocol.

Wallet Display

```
$ btcpc-cli balance shindevlin
```

```
BTCPC Balance: shindevlin
```

```
=====
```

```
Spendable:          972.22222220 BTCPC
Staked:             0.00000000 BTCPC
```

```
Genesis Dreams (transferable NFTs):
```

```
-----
```

#0	Block 0	"The Answer to the Ultimate Question..."	2026-03-25
#1	Block 1	[uninscribed]	2026-03-25
#2	Block 2	[uninscribed]	2026-03-25

```
Mining Proofs (soulbound, non-transferable):
```

```
-----
```

Block 0	243.06 BTCPC	qwen3.5:27b	2026-03-25
Block 1	243.06 BTCPC	qwen3.5:27b	2026-03-25
Block 2	243.06 BTCPC	qwen3.5:27b	2026-03-25

Transfer

```
$ btcpc-cli transfer-dream 0 --to alice
Enter password: *****
```

Transferred Genesis Dream #0 to alice
Original miner: shindevlin (permanently recorded)
Inscription: "The Answer to the Ultimate Question..."
Alice now owns this dream. Shindevlin retains Mining Proof #0.

Purpose-Built Contract Types (updated)

Contract Type	Required Key	Purpose
Inscribe	Active	Add a dream inscription to an owned genesis dream (once, permanent)
TransferDream	Active	Transfer a genesis dream to another account

Appendix G: Inscription Policy and Fee Structure

Inference: Fully Uncensored

BTCPD does not censor, filter, or inspect inference requests. All prompts and results are end-to-end encrypted. No node, validator, or protocol mechanism can read, block, or modify what users compute. This is a fundamental design principle — BTCPD is a compute utility, not a content platform.

Inference is private. Always. No exceptions.

Dream Inscriptions: Public Dream inscription Filtering

Genesis dream inscriptions are different — they are **public, plaintext dream inscriptions** visible on the block explorer and permanently on-chain. Because inscriptions are public-facing, the protocol applies automatic content filtering:

How it works: - When a requester submits dream inscription, the protocol scans the plaintext for prohibited patterns (CSAM-related terms, known abuse material identifiers) - Prohibited content is NOT rejected — it is **redacted in place**: matching text is replaced with XXXXXXXXXX - The inscription is still recorded, the dream is still created, the block is still valid - The requester's account is not banned or penalized - No human review, no censorship committee — purely automated pattern matching on public inscription text only

Submitted inscription:

"Building [prohibited content here] on BTCPD"

Recorded on-chain:

"Building XXXXXXXXXX on BTCPD"

The dream exists. The block is valid. The bad text is gone.

This approach: - **Does not censor inference** (encrypted, untouchable) - **Does not reject transactions** (the dream is still created) - **Does not punish users** (no bans, no slashing) - **Protects the public record** (block explorer stays clean)

Elevated Fee: External URLs

Inscriptions containing external URLs or links to outside websites incur an elevated fee:

Standard text inscription:	base_fee (e.g., 1 BTCPC)
Contains external URL/link:	base_fee × 10

Fee destination:

Standard:	100% to miner
Elevated:	50% to miner, 50% to BTCPC ops wallet

This prevents spam-linking and generates revenue for protocol development.

Genesis Dream Dream inscription Source

Dream inscriptions come from the REQUESTER, not the miner.

When a user submits an inference request, they can optionally include a dream inscription that will be recorded on the genesis dream of whichever block processes their request. The miner cannot modify this inscription — it is signed by the requester.

Inference request with dream inscription:

```
{
  "type": "inference_request",
  "requester": "alice",
  "model": "qwen3.5:27b",
  "encrypted_prompt": "...",
  "dream_inscription": {
    "project": "my-ai-app",
    "tag": "Building decentralized image generation"
  },
  "signature": "<signed by requester>"
}
```

- If this request is processed in block 42:
- Genesis Dream #42 inscription = alice's dream inscription
- Miner cannot alter it
- Alice's build is permanently recorded

Dream inscriptions are mandatory. Every inference request must include a dream inscription. If the requester does not provide a custom dream inscription, the protocol inserts a default:

Default dream inscription (auto-generated):

```
{
  "project": "btcpc-compute",
  "tag": "Inference request – [model] – [token count] tokens – epoch [N]"
}
```

This ensures every genesis dream has meaningful content. No empty dreams. Every block tells the story of what was computed.

BTCPC Foundation Wallet

A designated ops wallet receives elevated fee revenue (from URL inscriptions). This wallet is controlled by Shin Devlin and used for: - Protocol development funding - Security audit funding - Community grants - Bug bounty program

The ops wallet address will be designated in a future protocol update.

Appendix H: Multi-Provider CLI and External Compute Rewards

Universal AI Interface

The btcpc-cli supports multiple inference providers through a single interface. Users can bring their own API keys from any supported provider, or use BTCPC's native decentralized compute:

```
btcpc-cli inference --provider btcpc      "Explain quantum computing"
btcpc-cli inference --provider openai     "Explain quantum computing"
btcpc-cli inference --provider anthropic  "Explain quantum computing"
btcpc-cli inference --provider grok       "Explain quantum computing"
btcpc-cli inference --provider ollama     "Explain quantum computing"
```

One CLI. Any backend. Every request creates a dream inscription on the BTCPC chain.

External Provider Verification

External provider requests require no redundant computation. The provider's own API response serves as verification:

Provider response includes:

- Request ID (unique, queryable)
- Model name (provider-reported, not user-claimed)
- Token usage (provider-reported)
- Timestamp

This receipt is submitted to the BTCPC chain as proof the compute occurred.

The user cannot falsify it – the data comes from the provider's response, not the user's.

The 42% Rule

External provider compute follows **the 42% rule** — a completely separate reward structure from native compute:

NATIVE BTCPC COMPUTE (commit-reveal, 3 miners):

First miner:	50%	(fastest correct result)
Second miner:	30%	(verified the work)
Third miner:	20%	(verified the work)
OPS:	0%	
Total:	100%	→ all rewards go to miners who did the work

EXTERNAL PROVIDER COMPUTE (receipt-verified):

User:	42%	(brought the job + their API key)
OPS:	58%	(protocol development fund)
Miners:	0%	(did not perform the compute)
Total:	100%	→ no miner reward because no miner computed

Why this split works:

- **Miners earn nothing on external jobs** — they didn't do the compute. OpenAI did. BTCPC rewards only flow to those who contributed. This keeps mining rewards pure and honest.
- **Users earn 42%** — the answer. They brought real inference activity to the BTCPC chain using their own API key and their own money. That contribution is worth 42% of the reward.
- **The OPS wallet earns 58%** — the verification premium. In native compute, this 58% goes to the second and third verifying miners (30% + 20% + the remaining 8% implicit overhead). Since external provider jobs need no verification, this premium flows to the OPS wallet instead.
- **OPS fees come ONLY from external provider jobs.** Native compute pays zero to the foundation. This creates a clear incentive: mine natively and keep 100% within the miner community, or use external providers and fund protocol development.

The ops wallet, controlled by Shin Devlin, funds: - Protocol development - Security audits - Bug bounty programs - Community grants - Network infrastructure

The ops wallet address will be designated by Shin Devlin in a future protocol update.

Dream Types

Dreams created from external compute are marked differently on-chain:

Verified Dream:	backed by BTCPC native compute + commit-reveal proof
Registered Dream:	backed by external provider receipt

Both are valid dreams. Both carry inscriptions. Both are transferable. Verified dreams carry stronger provenance — proof of decentralized compute.

Supported Providers

Provider	Key Format	Models
BTCPC Native	btcpcli built-in	Any Ollama model on the network
OpenAI	OPENAI_API_KEY	gpt-4o, gpt-4, gpt-3.5-turbo
Anthropic	ANTHROPIC_API_KEY	claude-sonnet, claude-opus, claude-haiku
xAI (Grok)	XAI_API_KEY	grok-2, grok-3
Google	GOOGLE_API_KEY	gemini-pro, gemini-ultra
Local Ollama	OLLAMA_URL	Any locally installed model

The Funnel

The multi-provider CLI creates a natural conversion funnel:

Stage 1: User installs btcpcli to use their existing OpenAI key
→ Earns 42% rewards, creates dreams, sees the chain

Stage 2: User accumulates BTCPC from external compute rewards
→ Enough to explore native compute

Stage 3: User tries --provider btcpcli, earns 100% rewards
→ Realizes native compute is cheaper AND earns more

Stage 4: User installs Ollama, stakes 1000 BTCPC, becomes a miner
→ Full mining rewards + cross-chain bonuses + genesis dreams

Every stage creates token demand. Every stage generates dreams. Every stage grows the network.

API Key Security

API keys never leave the user's device. The btcpcli calls external providers directly from the user's machine:

User's device:

1. btcpcli calls OpenAI directly (key stays local)
2. OpenAI returns response + receipt
3. btcpcli submits receipt + dream inscription to BTCPC chain
4. No BTCPC node ever sees the API key

Security: identical to using the OpenAI SDK directly.

The BTCPC chain only sees: provider name, model, token count, receipt ID, dream inscription

Never: API key, prompt content, response content.

Appendix I: Native L2 — WASM Smart Contract Layer

Why L2 Not L1

Bitcoin's limitation is that adding smart contracts to L1 is nearly impossible without years of consensus and soft forks. Ethereum's mistake was putting smart contracts directly on L1 — resulting in congestion, high fees, and years spent building L2 rollups to fix it.

BTCPC avoids both problems by designing a **native L2 execution layer** from day 1:

BTCPC L1 (Settlement Layer):

- └ Transfers, staking, inference, dreams, mining
- └ Purpose-built contracts (hardcoded, fast, secure)
- └ Processes L2 state commitments
- └ Never runs arbitrary user code

BTCPC L2 (Execution Layer):

- └ WASM runtime — smart contracts in Rust, Go, JS, AssemblyScript
- └ Full programmability — DeFi, NFTs, DAOs, games
- └ Posts state roots to L1 every epoch
- └ Secured by L1 miners (same commit-reveal verification)

How It Works

1. **Developers deploy WASM contracts to L2** — compiled from Rust, Go, JavaScript, or any language that targets WebAssembly
2. **Users interact with L2 contracts** — send transactions, call functions, query state
3. **L2 nodes execute contracts** in a sandboxed WASM runtime
4. **Every epoch, L2 posts a state commitment to L1** — a single hash representing the entire L2 state
5. **L1 miners verify the state commitment** as part of their epoch work
6. **If disputed:** any node can challenge by re-executing the L2 transactions and proving a different state root (optimistic rollup model)

Architecture

User

- └ L1 transactions (transfers, staking, inference)
 - processed directly by L1 miners
- └ L2 transactions (contract calls, DeFi, NFTs)
 - processed by L2 execution nodes
 - state root posted to L1
 - L1 provides final settlement and security

L1 sees: one state_root hash per epoch from L2
L2 does: unlimited contract execution between epochs

What Miners Earn

BTCPC miners earn from three sources of useful work:

1. AI Inference compute (proof of compute – GPU work)
2. L2 contract execution (WASM execution – CPU work)
3. Transaction fees (L1 transfers, staking, claims)

All three are useful work. All three generate BTCPC rewards. The protocol does not distinguish between types of useful computation — a cycle spent on inference and a cycle spent executing a smart contract are both real work that someone paid for.

Plugin Architecture

The L1 transaction processor uses a plugin interface that makes L2 integration seamless:

Contract Interface (all contract types implement this):

```
validate(tx, state) → bool      // is this transaction valid?
execute(tx, state)  → newState  // apply the transaction
fee(tx)             → amount    // what does this cost?
```

Purpose-built contracts: pre-installed plugins (JavaScript)

L2 WASM contracts: user-deployed plugins (WebAssembly)

Same interface. Same validation. Same fee model.

New purpose-built contract types can be added to L1 via governance vote — no hard fork required. The plugin system means the protocol is extensible without redesigning the chain.

Timeline

- **Phase 0-2 (current):** Purpose-built contracts only. Plugin interface designed but WASM runtime dormant.
- **Phase 3:** WASM runtime activated via governance vote. Developers can deploy contracts to L2.
- **Phase 4:** L2 ecosystem matures. DEXs, lending, NFT marketplaces — all powered by BTCPC compute.

Why This Matters

BTCPC will never face Bitcoin's "we can't add features" problem. The L2 is native, designed from genesis, and governed by miners. When the ecosystem needs smart contracts, the runtime is already there — waiting to be activated. No years of debate. No contentious forks. Just a governance vote and the L2 goes live.

Appendix J: Fee Model — L1 Fixed Fees, L2 Gas

L1: No Gas. Fixed Fees.

BTCP L1 runs only purpose-built contracts with predictable execution costs. There is no gas metering, no gas estimation, and no failed transactions from running out of gas.

L1 Fee Schedule:

Transfer:	0.001 BTCP
Stake / Unstake:	0.001 BTCP
Inference request:	model-specific (based on tokens)
Dream inscription (text):	0.01 BTCP
Dream inscription (with URL):	0.1 BTCP (10x – elevated fee to OPS)
Account creation:	0.1 BTCP
Account update:	0.01 BTCP
Cross-chain claim:	0.001 BTCP
Dream transfer:	0.01 BTCP
MultiSig update:	0.01 BTCP
RecurringPay setup:	0.01 BTCP

Users always know exactly what they will pay. No surprises. No gas wars. Bitcoin-simple.

All L1 fees go to the miner who produced the block. Zero to OPS (except elevated URL inscription fees — 50% to miner, 50% to OPS).

L2: Gas. Mandatory.

WASM smart contracts are arbitrary code — a contract could loop forever, allocate unbounded memory, or perform complex computation. Gas metering is essential.

L2 Gas Model:

Unit:	1 gas = 1 WASM instruction
Price:	dynamic, adjusts with L2 demand (EIP-1559 style base fee)
Payment:	BTCP (same token as L1 – no separate gas token)
Gas limit:	set by user per transaction
Out of gas:	transaction reverts, gas consumed (standard model)
Minimum gas:	21,000 (matches Ethereum convention for simple operations)

One token. Two layers. No bridging.

Unlike Ethereum where you need ETH for gas AND a separate token for the dApp, BTCP uses one token everywhere. Users hold BTCP. They spend it on L1 operations (fixed fees) or L2 contracts (gas). No friction.

Fee Revenue Distribution

L1 fees:	100% to block-producing miner
L2 gas fees:	100% to WASM-executing miner
L1 URL fees:	50% miner / 50% OPS wallet

External provider: 42% user / 58% OPS wallet

Why No Gas on L1

Bitcoin does not have gas. Its transactions are simple and predictable. BTCPC L1 follows the same philosophy:

- Every L1 operation has a known, fixed cost
- No gas estimation required — wallets show exact fees before signing
- No transaction failures from gas miscalculation
- Simpler node implementation — no gas metering overhead on L1
- Better UX for non-technical users

Gas exists on L2 because L2 needs it — arbitrary code execution demands resource metering. L1 does not run arbitrary code, so L1 does not need gas.

Appendix K: L2 Gas Token — The Four-Way Value Engine

L2 Gas Token

The BTCPC L2 uses a dedicated gas token for WASM contract execution. This token is:
- **Purchased with stablecoins only** (USDC, USDT, DAI) - **Priced at \$1 = 1 GAS** (always, no speculation) - **Not mineable** — purchase only - **Burns on use** — deflationary by design

The Four-Way Split

Every L2 gas purchase and expenditure creates value in four directions simultaneously:

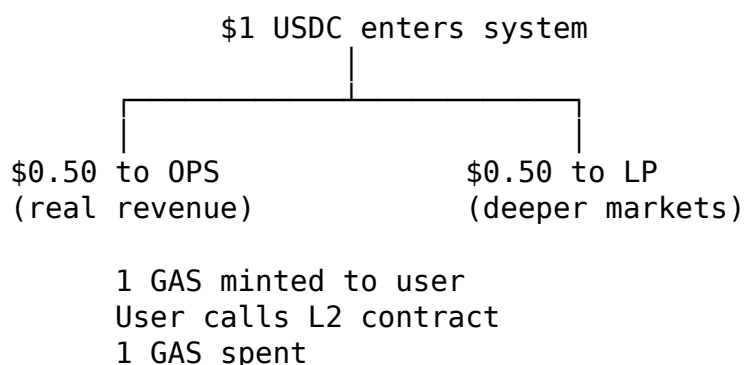
On Purchase (\$1 USDC → 1 GAS):

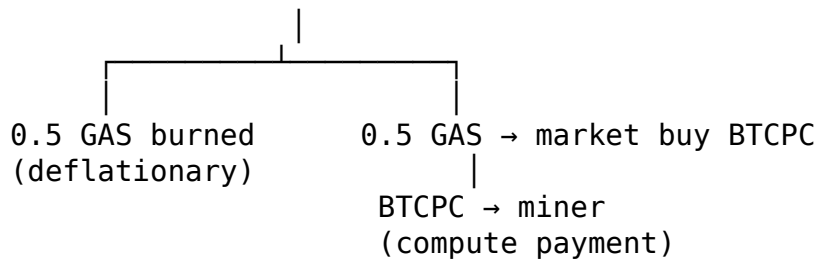
\$0.50 → OPS wallet (stablecoin operational revenue)
\$0.50 → BTCPC/USDC liquidity pool (deepens market depth)

On Use (1 GAS spent on contract call):

0.5 GAS → burned (permanently destroyed, deflationary)
0.5 GAS → buys BTCPC on market (constant buy pressure → pays miner)

Complete Flow





Economic Effects

OPS: Receives \$0.50 in stablecoins for every \$1 of L2 gas purchased. This is sustainable, non-volatile revenue — independent of BTCPC price. At 10,000 L2 transactions per day averaging \$0.10 gas each: \$500/day to OPS, \$182,500/year in stablecoin revenue.

Liquidity: \$0.50 of every gas purchase deepens the BTCPC/USDC trading pool. More L2 usage = deeper liquidity = tighter spreads = better price discovery = more institutional interest.

BTCP price: Every L2 contract call triggers a market buy of BTCP (from the 0.5 GAS conversion). This is constant, organic buy pressure that scales with L2 adoption. Not speculative — driven by real usage.

Miners: Receive BTCPC purchased at market rate for executing WASM contracts. They are paid in the same token they mine. Their incentive structure is unified across L1 inference and L2 contract execution.

Gas token supply: Half of all gas tokens ever minted are eventually burned. Circulating supply stays low. No secondary market speculation — the token is minted at \$1 and burned on use. It is a utility, not an investment.

The Flywheel

More L2 dApp usage

- [illegible]

Every component reinforces every other component. L2 adoption directly drives L1 token value, miner profitability, market liquidity, and operational sustainability — simultaneously.

Paying L2 Gas with BTCPC (Alternative)

Users who hold BTCPC can pay L2 gas directly in BTCPC at a **15% discount** versus the stablecoin price:

Via stablecoin: \$1.00 USDC → 1 GAS → four-way split

Via BTCPC: \$0.85 worth of BTCPC → 1 GAS → BTCPC distributed directly to miner

The 15% discount incentivizes holding BTCPC over stablecoins.

When paying with BTCPC directly, there is no stablecoin revenue to OPS and no liquidity pool contribution — the BTCPC goes directly to the executing miner. This is acceptable because BTCPC payment means the user is already part of the BTCPC ecosystem. The stablecoin pathway is for onboarding new users.

Appendix L: Governance and Decentralization — The Path to Irrelevance

The Founder's Duty

Satoshi Nakamoto built Bitcoin, mined alone, and walked away. The network thrived precisely because its creator became irrelevant. The protocol was the authority, not the person.

Shin Devlin follows the same path. The goal is not to build a company, a foundation, or an empire. The goal is to build a protocol that does not need its creator — and then to prove it by leaving.

Shin and Satoshi are brothers in philosophy: build something that outlives you, then let it.

The Five Stages of Decentralization

Stage 1: Genesis (current) Shin Devlin is the sole miner, sole developer, and sole authority. This is necessary — someone must write the first line of code, mine the first block, and make the first decisions. Every decentralized network begins as a centralized one. Bitcoin did. BTCPC does.

Stage 2: Delegation (at 10+ miners) Other miners join the network. Shin remains the lead developer but accepts contributions. Protocol discussions happen in public. The codebase has multiple contributors. Shin's authority comes from competence, not from a special key.

Stage 3: Governance (at 50+ miners) Stake-weighted governance activates. Protocol changes require a miner vote. Shin has a vote proportional to his stake — equal in weight to any other staker. The OPS wallet transitions from Shin's sole control to a multi-signature council.

Stage 4: Participation (at 1000+ miners) Shin is just another miner. No special permissions, no emergency powers, no veto. The OPS wallet is controlled by an elected council. The codebase is maintained by the community. Protocol evolution is pure stake-weighted democracy.

Stage 5: Legend Shin steps back. Perhaps gradually, perhaps all at once. The protocol does not need him. The network mines, verifies, and dreams without a founder at the helm. Like Satoshi before him, Shin's greatest contribution is proving that the system works without its creator.

The best thing a founder can do is make themselves unnecessary.

Governance Mechanism

Built into the protocol from genesis, activated at Stage 3:

Proposals:

Any account with ≥ 1000 BTCPC staked can submit a proposal.

Voting:

1 staked BTCPC = 1 vote

Voting period: 1000 epochs (~3.5 days)

Quorum: 20% of total staked BTCPC must participate

Approval threshold: 66% to pass

Proposal types:

- Protocol parameter changes (epoch length, fees, reward splits)
- Add new purpose-built contract type to L1
- Activate or upgrade the WASM L2 runtime
- Elect or remove OPS council members
- Modify the content redaction wordlist
- Emergency protocol pause (requires 80% supermajority)

No single account — including shindevin — has veto power. Governance is stake-weighted, one-token-one-vote, majority rules.

OPS Wallet Transition

Stage	Control	Structure
1 (Genesis)	Shin Devlin	Single key
2 (10+ miners)	Shin + 2 early contributors	2-of-3 multi-sig
3 (50+ miners)	Elected council	3-of-5 multi-sig
4 (1000+ miners)	Elected council	5-of-9 multi-sig
5 (Mature)	All stakers	DAO treasury — spending requires governance vote

At Stage 5, the OPS wallet is no longer controlled by any individual or council. Spending proposals are submitted, voted on by all stakers, and executed automatically by the protocol. No human gatekeeper.

Power Divestiture Schedule

Power	When Shin Relinquishes	How
Sole mining	Stage 2 (other miners join)	Natural — network opens
Protocol decisions	Stage 3 (governance activates)	Governance vote required for changes
OPS wallet control	Stage 3 → Stage 5 (gradual)	Multi-sig → DAO
Reserved names (420)	Stage 2-4 (sold/granted over time)	Market sales + community grants
Emergency pause	Stage 4 (80% supermajority required)	No single-key pause
All special authority	Stage 5	Nothing remains. Shin is a miner, nothing more.

The Satoshi Standard

Satoshi Nakamoto demonstrated that a pseudonymous creator can build the most valuable network in human history and walk away. The network's value did not depend on Satoshi's continued involvement — it depended on the protocol being sound.

BTCPC adopts this as a design principle: **the protocol is the authority, not the founder.**

Shin Devlin's roadmap mirrors Satoshi's: 1. Build the protocol 2. Mine the genesis blocks 3. Attract other miners 4. Transfer governance to the community 5. Step back

The day Shin Devlin walks away from BTCPC — and the network continues mining, verifying, and dreaming without interruption — is the day the project succeeds. Not the day the token reaches a target price. Not the day a certain number of miners join. The day the founder becomes irrelevant.

That is the goal. That has always been the goal.

"I think the problem, to be quite honest with you, is that you've never actually known what the question is."

The question was never "who controls the network?"

The answer, as always, is 42.